

Scaling Scientific Discovery Environments for Turn-Level Agentic RL

Yucheng Xu[†], Keyi Zhang[†], Yuyang Yu, Min Zhang,
Shiyuan Meng, Pei Chu, Zhongying Tu^{*}
Shanghai AI Lab

[†]Equal contribution ^{*}Corresponding author

Abstract

Large language model agents have shown promising capabilities in data-driven scientific discovery tasks, where an agent interacts with an execution environment and produces a statistical claim. Long-horizon scientific analysis remains constrained by the lack of process-supervised environments over real-world scientific data. This paper introduces **SciDisco**, a scalable framework for training **Scientific Discovery** agents in process-verifiable environments. **SciThèque** compiles hypotheses, datasets, hidden evidence graphs, and verifiers into task environments where analytical progress can be checked during interaction. DAG-grounded trajectory synthesis uses these environments to construct verifier-filtered multi-turn demonstrations. **DiscoPO** then uses the environment as the source of training signal, assigning turn-level credit to actions that produce verifiable analytical evidence. Experiments show that SciDisco-14B reaches state-of-the-art on hypothesis-driven scientific data analysis benchmarks.

1 Introduction

Data-driven scientific discovery is a hypothesis-driven mode of empirical research over existing scientific datasets. In human practice, it requires researchers to understand scientific context and dataset structure, select appropriate methods, execute statistical analyses, and draw verifiable conclusions. Existing LLMs and LLM-based agents remain unreliable in this setting. Prior work shows that models struggle with statistical method applicability and data-based causal reasoning (Zhu et al., 2024; Liu et al., 2024a). Agentic code and reasoning scaffolds also achieve limited performance on data-driven discovery benchmarks (Majumder et al., 2025; Gu et al., 2024; Chen et al., 2025). These results suggest that data-driven scientific discovery should not be treated as one-shot question answering or single-turn code generation, but as an

interactive process of executing, observing, revising, and verifying analyses over real data.

The core challenge is to train agents that learn robust interaction policies, rather than merely encode scientific knowledge or generate code. Prompts and hand-written scaffolds can describe the steps of an analysis workflow, but benchmark evidence suggests that prompting and generic multi-turn training do not reliably produce goal-directed interaction. In contrast, reward-trained tool use can outperform supervised tool traces in unfamiliar tool settings (Abdulhai et al., 2023; Qian et al., 2025; Wang et al., 2025b). Data-driven discovery agents must therefore learn from environmental feedback about when to inspect data, choose methods, test assumptions, revise analyses, and submit conclusions. This motivates training specialized discovery agents inside scientific data environments, rather than relying only on foundation models or inference-time workflow orchestration.

Such training requires environments that are both executable and verifiable. A discovery RL environment should provide data-driven goals, stateful code execution, observations, and automatic verification. Open scientific datasets are abundant, but they usually remain static files or metadata without task goals or verifiers. Existing scientific-discovery benchmarks provide valuable evaluation tasks and ground-truth criteria, but they do not expose hidden scientific evidence states for large-scale process-supervised training with persistent cross-turn state and verifier-grounded turn credit (Majumder et al., 2025; Gu et al., 2024; Chen et al., 2025). General agent and ML engineering environments support executable feedback and, in some cases, training, but their objectives target web interaction or ML engineering rather than real-data hypothesis verification (Liu et al., 2024b; Chan et al., 2024; Qiang et al., 2025; Zhou et al., 2026).

Executable environments alone are still not enough for effective RL. Without a behavioral prior,

early rollouts can be dominated by invalid tool calls, variable misreadings, and incomplete analyses. Data-driven scientific discovery also poses a long-horizon credit-assignment problem: final conclusions depend on schema understanding, method selection, code execution, diagnostics, and evidence interpretation, while outcome-only rewards cannot identify which turns produced valid analytical evidence (Wang et al., 2025b; Wei et al., 2025; Li et al., 2026b; Xie et al., 2026).

These limitations point to a gap in agent training: data-driven scientific discovery needs an interface in which analytical progress can be executed, verified, and credited during interaction. **SciDisco** provides such a framework for data-driven scientific discovery agents. Its environment layer, **SciThèque**, compiles open scientific datasets into verifiable hypothesis-driven environments. Hidden evidence DAGs support verified multi-turn demonstrations for SFT and provide process feedback that **DiscoPO** uses for turn-level credit assignment in agentic RL. SciDisco therefore aligns task construction, behavior imitation, and policy optimization within realistic scientific data environments.

The paper makes three contributions:

1. **SciThèque** builds scalable discovery environments from open scientific datasets with hypotheses, sandboxed execution, and verifiers.
2. DAG-grounded trajectory synthesis cold-starts agent behavior using verified multi-turn demonstrations.
3. **DiscoPO** assigns turn-level credit to verified evidence-producing processes instead of only final answers.

2 Related Work

2.1 Discovery Agents and Environments

Prior work on data-science and scientific-discovery agents follows two broad paradigms. The first improves data-specific capabilities, including tabular understanding, database operations, statistical reasoning, and quantitative reasoning over data (Zhu et al., 2024; Liu et al., 2024a). The second moves beyond single-turn QA by building agentic workflows with ReAct-style interaction, code interpreters, multi-agent scaffolds, or learned analysis trajectories. These systems often rely on strong closed-source models, leaving the underlying training environment difficult to reuse (Guo et al., 2024; Zhang et al., 2023; Hong et al., 2024; Li et al.,

2024; Zhang et al., 2025b; Qiao et al., 2025). This makes it difficult to separate model capability from the executable interaction and verification substrate used to produce scientific conclusions.

Discovery benchmarks evaluate agents on real data, open-ended questions, code execution, and conclusion checking (Majumder et al., 2025; Gu et al., 2024; Chen et al., 2025; Egg et al., 2025). Interactive science environments support exploration in text-based or virtual worlds (Wang et al., 2022; O’Sullivan et al., 2024). General agent and MLE environments cover tool use, web interaction, executable feedback, and training efficiency (Liu et al., 2024b; Zhou et al., 2024; Chan et al., 2024; Qiang et al., 2025; Zhou et al., 2026). These settings are valuable, but they are not built around real-data hypothesis verification with persistent execution state and verifier-grounded process feedback. SciThèque constructs executable, verifiable discovery environments that connect data, hypotheses, code execution, and reusable multi-turn feedback, so the same environment can define tasks, filter trajectories, and provide RL signals. This shifts the role of an environment from a static evaluation artifact to a reusable training interface for scientific agent learning.

2.2 Reinforcement Learning for LLM Agents

Beyond prompting and SFT, recent work trains LLM agents with reinforcement learning. GRPO-style RLVR uses group-relative advantages to avoid explicit critics and train models on verifiable reasoning or coding tasks (Shao et al., 2024; Guo et al., 2025a). Later variants improve sampling, clipping, normalization, length bias, and self-distillation routing (Yu et al., 2025; Liu et al., 2025; Li et al., 2026a). These methods provide a strong optimization backbone for agent training, but their rewards usually remain at the response or trajectory level.

For multi-turn agents, recent work assigns credit at step, turn, or segment granularity, or calibrates intermediate decisions with hindsight, reflection, or trajectory correction (Feng et al., 2025; Wei et al., 2025; Li et al., 2026b; Zong et al., 2026; Guo et al., 2025b; Lu et al., 2026; Wang et al., 2025a,c). These methods recognize that final rewards cannot identify which intermediate actions advanced the task. However, their signals often depend on repeated states, tree search, segmenters, hindsight models, reflection, or task-specific step rewards, rather than evidence produced by executed scientific analyses. DiscoPO instead uses scientific verifiers to credit

turns that produce validated analytical evidence, aligning turn-level RL with hypothesis verification.

3 Problem Formulation

A scientific discovery task is formulated as a hypothesis-driven data analysis problem. Given a dataset, the task is to evaluate a scientific hypothesis, produce statistical findings, and support those findings with executable analysis.

Let $\mathcal{U} = \{u_j\}_{j=1}^N$ denote a collection of such tasks. Each task is represented as:

$$u_j = (h_j, d_j, v_j), \quad (1)$$

where h_j specifies the hypothesis or analysis goal, d_j is the dataset associated with the task, and v_j denotes the task verifier.

Each task u_j is paired with an environment e_j , and the corresponding environment collection is $\mathcal{E} = \{e_j\}_{j=1}^N$. The environment exposes h_j and d_j , executes the agent’s actions in a sandbox, and returns observations, while task-internal verification state may remain hidden. The interaction between policy π_θ and environment e_j can therefore be modeled as a partially observable Markov decision process (POMDP), producing a trajectory τ_j :

$$\tau_j = (o_{j,0}, a_{j,1}, o_{j,1}, \dots, a_{j,T_j}, o_{j,T_j}), \quad (2)$$

where $o_{j,0}$ contains the initial task observation, actions $a_{j,t}$ can be executable analysis or terminal submission, and observations $o_{j,t}$ contain environment feedback.

A final reward provides only sparse credit assignment for a scientific discovery trajectory. It cannot distinguish an evidence-building trajectory from a one-shot shortcut that happens to produce the same submitted findings.

Let $r_{j,t}$ denote the process reward assigned to turn t , measuring whether that turn contributes verifiable progress toward the task findings. The training goal is to maximize the accumulated process reward:

$$R_j(\tau_j) = \sum_{t=1}^{T_j} r_{j,t}, \quad (3)$$

and the optimization objective is:

$$J(\theta) = \mathbb{E}[R_j(\tau_j)], \quad (4)$$

where the expectation is over tasks sampled from $\mathcal{U}$ and trajectories generated by π_θ in their paired environments.

4 Methodology

SciDisco turns the process-reward formulation in Section 3 into a three-stage post-training pipeline for scientific discovery agents. **SciThèque** compiles scientific datasets into sandboxed task environments with task-local hidden evidence DAGs. DAG-grounded trajectory synthesis uses these environments to produce verifiable multi-turn demonstrations for SFT cold-start. **DiscoPO** then optimizes the policy with turn-level credit from environment-verified scientific progress. The task-local hidden evidence DAG is the shared interface across these stages: it defines what can be synthesized, what counts as an accepted transition during online interaction, and how progress is credited during RL.

4.1 SciThèque

SciThèque instantiates the environment collection $\mathcal{E}$ defined in Section 3. It constructs tasks $u_j = (h_j, d_j, v_j)$ from source datasets and hypothesis templates, then mounts each task in a sandbox runtime to form an environment e_j . The agent sees h_j , data files d_j , and observations $o_{j,t}$, while the verifier logic, evidence DAG, and progress state remain hidden. Appendix A reports the data-source distribution and materialized environment contract.

Catalog construction. SciThèque first builds candidate dataset entries $c_i = (s_i, d_i, m_i, \rho_i)$, where s_i is the source, d_i the dataset, m_i the modality, and ρ_i a structural profile recording provenance, licence metadata, file structure, size, and analysis-relevant fields. Inexpensive gates remove unsuitable datasets before task creation, and each surviving dataset is classified as tabular, temporal, graph, spatial, or sequence data. Appendix A reports source citations and snapshot counts.

Template-guided hypothesis generation. For each modality m , SciThèque maintains a template library $\mathcal{T}_m$ indexed by statistical pattern and outcome type. Each catalog entry is matched to templates allowed by ρ_i , and an external LLM proposes constrained specifications $\mathcal{H}_i = \{(T_{i,k}, \eta_{i,k})\}_{k=1}^{K_i}$, where $T_{i,k} \in \mathcal{T}_{m_i}$ is an applicable template and $\eta_{i,k}$ instantiates it on d_i . The LLM proposes candidates but does not certify validity; retained specifications from all $\mathcal{H}_i$ form the materialization pool.

Environment materialization. Materialization turns each retained specification into a training environment. SciThèque runs the reference analysis on d_i under the selected template and accepts

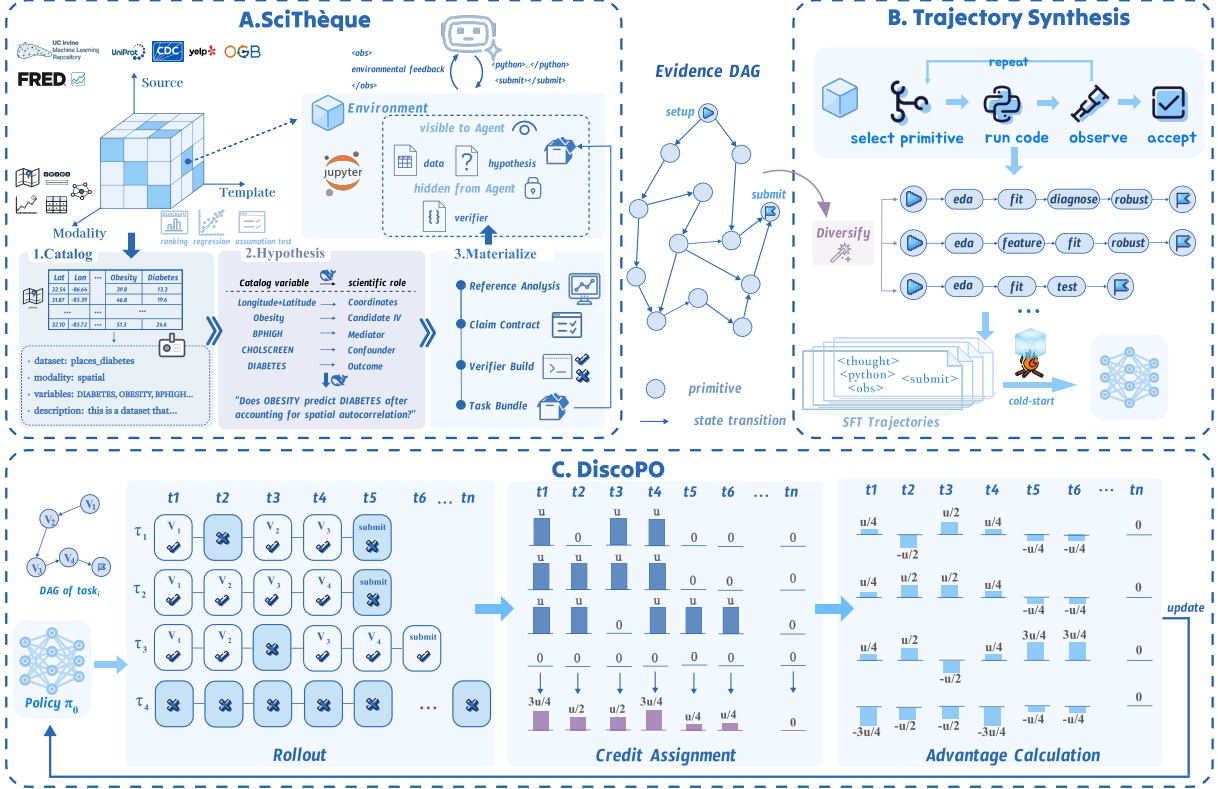


Figure 1: The pipeline of **SciDisco**: 1) **SciThèque** builds a scalable collection of data–hypothesis–verifier environments from scientific datasets; 2) **Trajectory Synthesis** traverses the evidence DAG through executable analysis primitives and keeps accepted state transitions as multi-turn SFT demonstrations; 3) **DiscoPO** defines turn-level rewards by increases in verified DAG progress, assigning credit to evidence-producing turns.

the specification only if the analysis executes and yields verifier-backed findings. The accepted specification becomes $u_j = (h_j, d_j, v_j)$, then is paired with sandbox runtime S_j to form $e_j = (u_j, S_j)$.

The verifier v_j is grounded in a hidden evidence DAG:

$$g_j = (\mathcal{V}_j, \mathcal{A}_j), \quad (5)$$

Each node in $\mathcal{V}_j$ defines a verifiable scientific state transition and is typed by a process primitive, such as data inspection, model fitting, diagnostic checking, or robustness analysis. The primitive type determines the admissible family of analysis operations for that node, so a turn is verified against the scientific role of the transition rather than arbitrary code execution. The directed edges $\mathcal{A}_j$ encode prerequisite relations, so only frontier nodes are eligible at a given turn. The DAG is hidden from the agent and defines the progress state used by v_j to verify intermediate work. Appendix B details the visibility split, evidence-graph record, progress rule, and leakage controls.

To assess whether the verifier design matches domain-level scientific practice, 10 domain experts covering the scientific domains represented in the

environment corpus manually reviewed the evidence primitives, prerequisite structure, and acceptance criteria used by the hidden evidence graphs. Appendix B reports the review protocol and audit scope.

The materialized output is therefore an environment specification, not a static question-answer pair. Each task stores its data files, instructions, verifier, task-local evidence DAG, final checker, and reproducibility metadata.

4.2 DAG-Grounded Trajectory Synthesis

DAG-grounded trajectory synthesis uses SciThèque environments to construct SFT demonstrations for model cold-start. Although the evidence DAG defines verifiable scientific state transitions, a base policy may not know how to realize them as executable analysis turns. SFT therefore places the policy in the right interaction regime before RL begins. Appendix C reports the synthesis stages, filtering criteria, and retained corpus snapshot.

For each materialized task u_j with hidden evidence DAG g_j , the synthesizer constructs a pri-

vate transition schedule $\sigma_{j,\ell}$ over the required DAG nodes. This schedule specifies the verified transition executed at each step while respecting the prerequisite structure of g_j . For each scheduled node, the synthesizer selects an executable program variant consistent with the node’s process primitive. The schedule and program choices remain hidden. The transcript exposes only assistant reasoning `<thought>`, code actions `<python>`, sandbox observations `<obs>`, and a final `<submit>` action.

Each synthesized trajectory uses a fresh sandbox runtime, where the scheduled `<python>` actions are executed. After each execution, the verifier v_j checks whether the visible observation establishes the scheduled transition in g_j . A trajectory is rejected if the code fails, produces no verifiable evidence, matches the wrong frontier node, violates the primitive process constraint, or accesses hidden verifier files. Each retained SFT turn therefore corresponds to an accepted scientific state transition rather than an arbitrary code fragment.

After all scheduled transitions are accepted, synthesis closes the transcript with a `<submit>` action. The submission records findings already established through verified transitions rather than introducing a separate terminal reward. Let $\mathcal{T}_{\text{syn}}$ be the pool of candidate trajectories produced by synthesis, and let $\tau_{j,\ell}$ denote the ℓ -th candidate trajectory for task u_j . A candidate is retained in $\mathcal{D}_{\text{SFT}}$ only when the required nodes in g_j have been completed and the submitted findings are consistent with the accumulated verifier state:

$$\mathcal{D}_{\text{SFT}} = \{\tau_{j,\ell} \in \mathcal{T}_{\text{syn}} \mid A_j(\tau_{j,\ell}, g_j) = 1\}. \quad (6)$$

Here $A_j(\tau_{j,\ell}, g_j) = 1$ indicates that the trajectory completes the required DAG transitions in a valid order and submits the findings verified during those transitions.

For the same task, synthesis can retain several verifier-valid trajectories rather than a single canonical solution path. These trajectories may differ in transition schedule, executable program variant, finding-reporting order, and reasoning style, while sharing the same hidden DAG and verifier constraints. Training-data conversion removes scheduler labels, hidden node identifiers, reference values, verifier tolerances, and generator-only helper variables. This cold-start stage teaches the policy to reason through a scientific state-transition chain. DiscoPO can then optimize verified scien-

tific progress from a policy that already follows the basic scientific behavior.

4.3 DiscoPO

DiscoPO trains the policy in SciThèque environments. At each turn within a trajectory, the agent emits either a `<python>` cell or a `<submit>` action. The sandbox presents each task as an agent workspace, and its persistent kernel lets later turns reuse earlier variables, models, and files. Each observation provides evidence for verifying the next state transition.

GRPO computes a group-normalized advantage from trajectory-level rewards and broadcasts that advantage to all generated tokens of the trajectory (Shao et al., 2024). DiscoPO keeps the same group-relative normalization but computes the advantage from verifier-gated progress at each interaction turn.

Algorithm 1 DiscoPO

Require: task environments $\{e_j\}$, hidden DAGs $\{g_j\}$, old policy π_{old} , group size G , clip parameters $\epsilon_{\text{low}}, \epsilon_{\text{high}}$

- 1: **for** each training iteration **do**
- 2: sample a batch of task environments
- 3: **for** each sampled environment e_j **do**
- 4: sample G rollouts $\{\tau_{j,k}\}_{k=1}^G$ with π_{old}
- 5: **for** each rollout $k = 1, \dots, G$ **do**
- 6: initialize $C_{j,k,0} \leftarrow \emptyset$
- 7: **for** each turn t in $\tau_{j,k}$ **do**
- 8: use v_j to update $C_{j,k,t+1}$ from $C_{j,k,t}$
- 9: compute $r_{j,k,t}$ by Eq. 8
- 10: **end for**
- 11: **end for**
- 12: compute $b_{j,t}$ and $A_{j,k,t}$ by Eqs. 11 and 12
- 13: assign $A_{j,k,t}$ to generated tokens in turn t and mask observations
- 14: **end for**
- 15: update π_θ with the clipped surrogate over turn-level advantages
- 16: set $\pi_{\text{old}} \leftarrow \pi_\theta$
- 17: **end for**

Verifier-gated state transitions. For task u_j , let $g_j = (\mathcal{V}_j, \mathcal{A}_j)$ be the hidden evidence DAG, and let $\mathcal{V}_j^{\text{exec}} \subseteq \mathcal{V}_j$ denote the required DAG nodes verified through `<python>` turns. For rollout k of task j , $C_{j,k,t}$ records the DAG nodes credited before turn t . Here j follows Section 4.1 and indexes the materialized task, while k indexes one trajectory in the rollout group used for group-relative advantage. Appendix F shows an example rollout trajectory.

At each turn, the verifier v_j considers only frontier nodes, meaning uncredited nodes whose prerequisites have already been completed. The verifier then checks whether the current `<python>` turn matches the process primitive assigned to one fron-

tier node. A turn is accepted only when the observation provides evidence for exactly one such node and the executed code satisfies its node-local process check.

Turns are rejected when they provide no observable evidence, match no frontier node, match multiple frontier nodes, repeat completed work, or collapse several analyses into one action. If a turn is accepted, the matched node is added to the hidden completed-node state; otherwise, the state is left unchanged. The `<submit>` action is checked against the verifier and can be accepted only after the required nodes have been credited. Appendix G gives concrete rejection categories and their training consequences.

Credit assignment. For task u_j , let $N_j = |\mathcal{V}_j^{\text{exec}}|$ be the number of required executable primitive transitions, and let s_j^{sub} denote the terminal submit node. The set of progress nodes is $\mathcal{P}_j = \mathcal{V}_j^{\text{exec}} \cup \{s_j^{\text{sub}}\}$. DiscoPO defines the DAG-progress potential over credited-node sets as:

$$\Phi_j(C) = \frac{|C \cap \mathcal{P}_j|}{|\mathcal{P}_j|}. \quad (7)$$

The per-turn process reward is the potential difference:

$$r_{j,k,t} = \Phi_j(C_{j,k,t+1}) - \Phi_j(C_{j,k,t}). \quad (8)$$

When a `<python>` turn is accepted, it adds one previously uncredited primitive node. A valid `<submit>` action adds the terminal submit node once all required primitives have been completed. Turns that do not add a new progress node receive zero reward. Equivalently:

$$r_{j,k,t} = \begin{cases} \frac{1}{N_j+1}, & \text{accepted primitive/submit,} \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

For each task environment, a group of G rollouts $\{\tau_{j,k}\}_{k=1}^G$ is sampled from the old policy π_{old} . Let $T_{j,k}$ be the length of rollout k . Rewards from terminated rollouts are zero-padded when computing the same-turn baseline:

$$\tilde{r}_{j,k,t} = \begin{cases} r_{j,k,t}, & t \leq T_{j,k}, \\ 0, & t > T_{j,k}. \end{cases} \quad (10)$$

This same-turn baseline compares whether rollouts for the same task make verifier progress at interaction index t ; it does not require different rollouts to complete the same DAG node at that index. DiscoPO then applies turn group normalization. The

same-turn baseline is:

$$b_{j,t} = \frac{1}{G} \sum_{k=1}^G \tilde{r}_{j,k,t}. \quad (11)$$

The turn advantage is:

$$A_{j,k,t} = \tilde{r}_{j,k,t} - b_{j,t}. \quad (12)$$

Policy optimization. The policy update assigns $A_{j,k,t}$ to policy-generated tokens in turn t , masks observation tokens, and applies a PPO clipped surrogate at turn granularity (Schulman et al., 2017). Let $\mathcal{I}_{j,k,t}$ denote the generated-token positions in that turn, let $\rho_i(\theta)$ be the old-policy likelihood ratio for token i , and let $\bar{\rho}_i(\theta)$ be its clipped value. DiscoPO optimizes:

$$\mathcal{L}_{\text{DiscoPO}}(\theta) = -\mathbb{E}_{j,k,t} \left[\frac{1}{|\mathcal{I}_{j,k,t}|} \sum_{i \in \mathcal{I}_{j,k,t}} s_i(\theta) \right],$$

$$s_i(\theta) = \min(\rho_i(\theta) A_{j,k,t}, \bar{\rho}_i(\theta) A_{j,k,t}). \quad (13)$$

The optimizer is standard; the change is that verifier-gated DAG progress supplies turn-level token weights instead of a scalar trajectory reward. Groups whose turn advantages are identically zero are filtered before optimization because they provide no group-relative policy-gradient signal. Appendix E expands the token-level notation.

5 Experiments

The experiments evaluate whether SciDisco improves data-driven scientific discovery and executable scientific data analysis. The evaluation covers three benchmark regimes: hypothesis generation over scientific datasets, multi-step business data analysis, and programmatically checked data-science workflows.

5.1 Benchmarks

DiscoveryBench. DiscoveryBench tests whether an agent can produce data-driven scientific hypotheses from a dataset and a discovery goal (Majumder et al., 2025). Table 1 reports the average Hypothesis Match Score (HMS), which aligns predicted and gold sub-hypotheses along context, variable, and relationship dimensions. GPT-5 Mini was used as the judge model during evaluation.

DABStep. DABStep contains 450 realistic multi-step data-analysis tasks over business datasets, split into 72 easy and 378 hard cases (Egg et al.,

Model	DiscoveryBench	DABStep			DataSciBench	
	Average HMS	Easy	Hard	Average	Success Rate	Completion Rate
<i>Proprietary Models</i>						
GPT-4o	13.2	66.7	6.1	15.8	66.3	68.4
GPT-5 Mini	26.9	68.1	15.3	23.8	45.9	63.4
Claude-Sonnet-4	23.2	81.9	19.8	29.7	67.6	76.6
<i>Open-source Models</i>						
DeepSeek-V4-Flash	28.3	79.2	12.2	22.9	60.8	72.1
Intern-S1-Pro	34.7	65.3	4.8	14.5	59.5	67.6
Qwen3-8B	11.3	47.2	3.2	10.2	31.5	48.4
Qwen3-14B	20.0	55.6	4.2	12.4	32.4	45.9
Qwen3-32B	24.1	59.7	4.5	13.3	36.0	52.4
<i>Related Methods</i>						
DeepAnalyze-8B	25.9	70.8	32.8	38.9	46.8	57.3
AutoSDT-Coder-14B	7.3	56.9	2.7	11.3	1.0	1.3
DataMind-14B	25.5	68.1	3.2	13.6	20.3	24.8
SciDisco-14B	35.2	62.5	9.3	17.8	56.2	61.0

Table 1: Main results on DiscoveryBench, DABStep, and DataSciBench. Performance is reported as percentages. The best results for each model group are highlighted in bold.

2025). Its tasks emphasise business data analysis, where an agent completes operational analyses over enterprise-like tables, reports, visualisations, or decision-support queries. Table 1 reports easy accuracy, hard accuracy, and the task-count-weighted average.

DataSciBench. DataSciBench evaluates end-to-end data-science workflows with programmatic Task–Function–Code checks over data preparation, analysis, modelling, visualisation, and insight tasks (Zhang et al., 2025a). Table 1 reports the benchmark-level success rate and completion rate, while Table 2 further uses the DataSciBench breakdown to report fine-grained F1–F5 scores for data preparation, plot validity, data exploration, data visualization, and data modeling.

5.2 Models and Baselines

We organize baselines into three groups to separate general model capability from data-science-specific training. The proprietary group includes GPT-4o (OpenAI, 2024), GPT-5 Mini (OpenAI, 2025), and Claude Sonnet 4 (Anthropic, 2025), representing strong closed-source agentic models. The open-source group includes DeepSeek-V4-Flash (DeepSeek-AI, 2026), Intern-S1-Pro (Intern-S1-Pro Team, 2026), and Qwen3-8B/14B/32B (Yang et al., 2025), allowing comparison against publicly available general models at different scales. The related-method group includes DeepAnalyze-8B (Zhang et al., 2025b), AutoSDT-Coder-14B (Li et al., 2025), and DataMind-14B (Qiao et al., 2025), which are specialized for data analysis or scientific discovery. SciDisco-14B

is trained from Qwen3-14B using the SciDisco trajectory corpus and DiscoPO objective.

5.3 Experimental Setup

Starting from Qwen3-14B, we first apply cold-start SFT on SciDisco-produced verified trajectories and then perform DiscoPO agentic RL. Training uses the Slime post-training framework (THUDM, 2025), with SGLang for rollout generation and inference (Zheng et al., 2024). For the RL stage, we use a rollout batch size of 16 and a group size of 8. The resulting SciDisco-14B checkpoint is evaluated on the three benchmarks above. All training and inference are conducted on NVIDIA A100 GPUs; detailed hyperparameters, source licenses, and corpus distribution are reported in Appendix A and Appendix D.

5.4 Main Results

Table 1 compares proprietary, open-source, and specialised data-science agents across three scientific data-analysis benchmarks. SciDisco-14B obtains the strongest DiscoveryBench result among all listed models, reaching 35.2% average HMS. This score exceeds the strongest proprietary baseline, GPT-5 Mini at 26.9%, the strongest open-source baseline, Intern-S1-Pro at 34.7%, and the strongest related-method baseline, DeepAnalyze-8B at 25.9%.

SciDisco-14B also performs strongly on DataSciBench. It reaches 56.2% Success Rate and 61.0% Completion Rate, outperforming the related-method baselines on both metrics. The result remains below the strongest proprietary and open-

Setting	Coarse-grained Metrics		Fine-grained Metrics					
	Success Rate	Completion Rate	VLM	F1: Data Preparation	F2: Plot Validity	F3: Data Exploration	F4: Data Visualization	F5: Data Modeling
Qwen3-14B	32.4	45.9	/	16.1	41.1	29.3	23.6	22.3
+ SFT	46.8	53.3	/	37.9	51.8	32.8	50.9	23.4
+ SFT + GRPO	49.5	55.4	/	39.8	53.6	34.5	52.7	26.3
+ SFT + DiscoPO	56.3	61.0	/	43.0	57.1	41.4	56.4	31.6

Table 2: Ablation study on SciDisco training objectives. Performance is evaluated on DataSciBench. Success Rate reports pass rate, Completion Rate reports completion accuracy, and F1–F5 measure fine-grained performance across data-science sub-tasks. VLM was left empty because the backbone model lacks vision capability.

source models on DataSciBench, but closes much of the gap while using a 14B open-weight backbone.

DABStep shows a different pattern. SciDisco-14B reaches 17.8% average accuracy, below DeepAnalyze-8B at 38.9%, Claude-Sonnet-4 at 29.7%, and GPT-5 Mini at 23.8%. The hard split remains challenging for all Qwen3-family models, with Qwen3-8B, Qwen3-14B, and Qwen3-32B all below 5%. SciDisco improves over the Qwen3-14B backbone on DABStep, but does not match methods specialised for this benchmark regime.

5.5 Ablation Studies

Table 2 isolates the effect of the training objective on the 14B backbone using DataSciBench. SFT provides the main first-stage gain over Qwen3-14B, and adding GRPO gives a further but smaller improvement. The full SFT + DiscoPO setting obtains the strongest coarse-grained result, reaching 56.3% Success Rate and 61.0% Completion Rate.

The fine-grained metrics follow the same ordering. SFT + DiscoPO obtains the highest score on every reported sub-task, with the largest incremental gains over SFT + GRPO appearing in Data Modeling and Data Exploration. These two categories are the clearest ablation signal for the turn-level objective, while the detailed mechanism is analysed in Section 5.6.

5.6 Analysis

The results support a specific reading of SciDisco’s contribution. DiscoveryBench is closest to the paper’s data-driven scientific discovery setting because it evaluates hypothesis construction over scientific datasets. SciDisco-14B’s leading HMS score indicates that training on process-verifiable scientific environments improves the model’s ability to form benchmark-matched scientific hypotheses from data.

DataSciBench provides a complementary exe-

cutable analysis test. The gains over related methods and the ablation improvements in Table 2 indicate that SciDisco also improves code-mediated data analysis under programmatic checks. The strongest ablation gains appear in Data Exploration and Data Modeling, the two categories most directly tied to intermediate analytical decisions.

DABStep exposes the boundary of the current training distribution. Its business data-analysis tasks often require completing operational analyses, reports, visualisations, or decision-support queries over enterprise-like tables. SciDisco instead trains on hypothesis-driven scientific environments where progress is defined by verifier-grounded evidence for statistical claims. The weaker DABStep result therefore suggests that the current environment distribution transfers better to scientific hypothesis analysis and executable scientific data analysis than to all forms of multi-step business analytics.

The ablation results locate where the training signal enters. SFT gives the largest first-stage gain by exposing the model to verifier-filtered executable trajectories. GRPO adds a smaller improvement over SFT, indicating that trajectory-level outcome optimisation supplies useful reinforcement. SFT + DiscoPO gives the strongest result because the reward is assigned to turns that complete hidden evidence-state transitions, rather than only to trajectories that end with a correct answer.

6 Conclusion

We presented SciDisco, a framework for training data-driven scientific discovery agents in process-verifiable environments. Hidden evidence DAGs and verifier checks make intermediate analytical progress trainable, while SciTh‘eque compiles scientific datasets and hypothesis templates into sandboxed environments shared by DAG-grounded synthesis and DiscoPO. Evaluation results support SciDisco as a scalable pipeline for converting verified analytical evidence into process supervision.

Limitations

SciDisco focuses on executable data-analysis workflows over text-readable scientific data, rather than wet-lab discovery, multimodal scientific artifacts, or open-ended hypothesis generation beyond the template space. The coverage of the environment distribution is also limited by the available source adapters, modality handlers, and hypothesis templates. Finally, environment-accepted progress is still a proxy for scientific quality: it verifies whether an analysis step was valid for the task contract, but it does not by itself establish that the agent made a novel scientific discovery.

References

- Marwa Abdulhai, Isadora White, Charlie Snell, Charles Sun, Joey Hong, Yuexiang Zhai, Kelvin Xu, and Sergey Levine. 2023. [LMRL gym: Benchmarks for multi-turn reinforcement learning with language models](#). *arXiv preprint arXiv:2311.18232*.
- Anthropic. 2025. [Introducing Claude 4](#). Anthropic News.
- Centers for Disease Control and Prevention. 2026a. [Flu-View interactive](#). Public health data service. Accessed 25 May 2026.
- Centers for Disease Control and Prevention. 2026b. [PLACES: Local data for better health](#). Public health data service. Accessed 25 May 2026.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lillian Weng, and Aleksander Madry. 2024. [MLE-bench: Evaluating machine learning agents on machine learning engineering](#). *arXiv preprint arXiv:2410.07095*.
- Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. 2025. [ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery](#). In *International Conference on Learning Representations*.
- DeepSeek-AI. 2026. [DeepSeek-V4-Flash](#). Hugging Face Model Card.
- Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. 2025. [DABstep: Data agent benchmark for multi-step reasoning](#). *arXiv preprint arXiv:2506.23719*.
- Federal Reserve Bank of St. Louis. 2026. [FRED, federal reserve economic data](#). Data service. Accessed 25 May 2026.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. [Group-in-group policy optimization for LLM agent training](#). *arXiv preprint arXiv:2505.10978*.
- Rakshitha Godahewa, Christoph Bergmeir, Geoffrey I. Webb, Rob J. Hyndman, and Pablo Montero-Manso. 2021. [Monash time series forecasting archive](#). In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.
- Ken Gu, Ruoxi Shang, Ruien Jiang, Keying Kuang, Richard-John Lin, Donghe Lyu, Yue Mao, Youran Pan, Teng Wu, Jiaqian Yu, Yikun Zhang, Tianmai M. Zhang, Lanyi Zhu, Mike A. Merrill, Jeffrey Heer, and Tim Althoff. 2024. [BLADE: Benchmarking language model agents for data-driven science](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025a. [DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning](#). *arXiv preprint arXiv:2501.12948*.
- Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. [DS-Agent: Automated data science by empowering large language models with case-based reasoning](#). In *International Conference on Machine Learning*.
- Yiran Guo, Lijie Xu, Jie Liu, Dan Ye, and Shuang Qiu. 2025b. [Segment policy optimization: Effective segment-level credit assignment in RL for large language models](#). *arXiv preprint arXiv:2505.23564*.
- Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Chenxing Wei, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Xiangru Tang, and 8 others. 2024. [Data interpreter: An LLM agent for data science](#). *arXiv preprint arXiv:2402.18679*.
- Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. [Open graph benchmark: Datasets for machine learning on graphs](#). In *Advances in Neural Information Processing Systems*.
- Intern-S1-Pro Team. 2026. [Intern-S1-Pro: Scientific multimodal foundation model at trillion scale](#). *arXiv preprint arXiv:2603.25040*.
- Markelle Kelly, Rachel Longjohn, and Kolby Nottingham. 2023. [The UCI machine learning repository](#). Data repository.
- Gengsheng Li, Tianyu Yang, Junfeng Fang, Mingyang Song, Mao Zheng, Haiyun Guo, Dan Zhang, Jinqiao Wang, and Tat-Seng Chua. 2026a. [Unifying group-relative and self-distillation policy optimization via sample routing](#). *arXiv preprint arXiv:2604.02288*.

- Peiji Li, Linyang Li, Handa Sun, Wenjin Mai, Yongkang Chen, Xiaozhe Li, Yue Shen, Yichuan Ma, Yiliu Sun, Jiaxi Cao, Zhishu He, Bo Wang, Xiaoqing Zheng, Zhaori Bi, Xipeng Qiu, Qipeng Guo, Kai Chen, and Dahua Lin. 2026b. [TL-GRPO: Turn-level RL for reasoning-guided iterative optimization](#). *arXiv preprint arXiv:2601.16480*.
- Yifei Li, Hanane Nour Moussa, Ziru Chen, Shijie Chen, Botao Yu, Mingyi Xue, Benjamin Burns, Tzu-Yao Chiu, Vishal Dey, Zitong Lu, Chen Wei, Qianheng Zhang, Tianyu Zhang, Song Gao, Xuhui Huang, Xia Ning, Nesreen K. Ahmed, Ali Payani, and Huan Sun. 2025. [AutoSDT: Scaling data-driven discovery tasks toward open co-scientists](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 30396–30418.
- Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, Wanjun Zhong, Wangchunshu Zhou, Wenhao Huang, and Ge Zhang. 2024. [AutoKaggle: A multi-agent framework for autonomous data science competitions](#). *arXiv preprint arXiv:2410.20424*.
- Xiao Liu, Zirui Wu, Xueqing Wu, Pan Lu, Kai-Wei Chang, and Yansong Feng. 2024a. [Are LLMs capable of data-based statistical and causal reasoning? benchmarking advanced quantitative reasoning with data](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9215–9235.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, and 1 others. 2024b. [Agent-Bench: Evaluating LLMs as agents](#). In *International Conference on Learning Representations*.
- Zichen Liu, Chang Liu, Xueqing Liu, Daya Guo, Jun Chen, and Chunting Zhou. 2025. [Understanding R1-Zero-like training: A critical perspective](#). *arXiv preprint arXiv:2503.20783*.
- Zhicong Lu, Zichuan Lin, Wei Jia, Changyuan Tian, Deheng Ye, Peiguang Li, Li Jin, Nayu Liu, Guanguan Xu, and Wei Feng. 2026. [HISR: Hindsight information modulated segmental process rewards for multi-turn agentic reinforcement learning](#). *arXiv preprint arXiv:2603.18683*.
- Bodhisattwa Prasad Majumder, Harshit Surana, Dhruv Agarwal, Bhavana Dalvi Mishra, Abhijeetsingh Meena, Aryan Prakhari, Tirth Vora, Tushar Khot, Ashish Sabharwal, and Peter Clark. 2025. [DiscoveryBench: Towards data-driven discovery with large language models](#). In *The Thirteenth International Conference on Learning Representations*.
- OpenAI. 2024. [GPT-4o system card](#). *arXiv preprint arXiv:2410.21276*.
- OpenAI. 2025. [Introducing GPT-5 for developers](#). OpenAI Blog.
- John O’Sullivan, Alan Lindsay, Brian Magerko, Antonio Lieto, and Ashok K. Goel. 2024. [DiscoveryWorld: A virtual environment for developing and evaluating automated scientific discovery agents](#). *arXiv preprint arXiv:2406.06769*.
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tur, Gokhan Tur, and Heng Ji. 2025. [ToolRL: Reward is all tool learning needs](#). *arXiv preprint arXiv:2504.13958*.
- Rushi Qiang, Yuchen Zhuang, Yinghao Li, V K Dingu Sagar, Rongzhi Zhang, Changhao Li, Ian Shu-Hei Wong, Sherry Yang, Percy Liang, Chao Zhang, and Bo Dai. 2025. [MLE-Dojo: Interactive environments for empowering LLM agents in machine learning engineering](#). In *Advances in Neural Information Processing Systems*.
- Shuofei Qiao, Yanqiu Zhao, Zhisong Qiu, Xiaobin Wang, Jintian Zhang, Zhao Bin, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2025. [Scaling generalist data-analytic agents](#). *arXiv preprint arXiv:2509.25084*.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. [Proximal policy optimization algorithms](#). *arXiv preprint arXiv:1707.06347*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *arXiv preprint arXiv:2402.03300*.
- Damian Szklarczyk, Rebecca Kirsch, Mikaela Koutrouli, Katerina Nastou, Farrokh Mehryary, Rayan Hachilif, Annika L. Gable, Tao Fang, Nadezhda T. Doncheva, Sampo Pyysalo, Peer Bork, Lars J. Jensen, and Christian von Mering. 2023. [The STRING database in 2023: Protein-protein association networks and functional enrichment analyses for any sequenced genome of interest](#). *Nucleic Acids Research*, 51(D1):D638–D646.
- The UniProt Consortium. 2025. [UniProt: The universal protein knowledgebase in 2025](#). *Nucleic Acids Research*, 53(D1):D609–D617.
- THUDM. 2025. [slime: An SGLang-native post-training framework for RL scaling](#). Software repository.
- U.S. Geological Survey. 2026. [USGS water data for the nation](#). Earth-science data service. Accessed 25 May 2026.
- Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2013. [OpenML: Networked science in machine learning](#). *SIGKDD Explorations*, 15(2):49–60.
- Hanlin Wang, Jian Wang, Chak Tou Leong, and Wenjie Li. 2025a. [STeCa: Step-level trajectory calibration for LLM agent learning](#). In *Findings of the Association for Computational Linguistics: ACL 2025*.

- Ruoyao Wang, Peter Jansen, Marc-Alexandre Cote, and Prithviraj Ammanabrolu. 2022. [ScienceWorld: Is your agent smarter than a 5th grader?](#) *arXiv preprint arXiv:2203.07540*.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. 2025b. [RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning](#). *arXiv preprint arXiv:2504.20073*.
- Ziliang Wang, Xuhui Zheng, Kang An, Cijun Ouyang, Jialu Cai, Yuhang Wang, and Yichao Wu. 2025c. [StepSearch: Igniting LLMs search ability via step-wise proximal policy optimization](#). *arXiv preprint arXiv:2505.15107*.
- Quan Wei, Siliang Zeng, Chenliang Li, William Brown, Oana Frunza, Wei Deng, Anderson Schneider, Yuriy Nevmyvaka, Yang Katie Zhao, Alfredo Garcia, and Mingyi Hong. 2025. [Reinforcing multi-turn reasoning in LLM agents via turn-level reward design](#). *arXiv preprint arXiv:2505.11821*.
- Yutao Xie, Nathaniel Thomas, Nicklas Hansen, Yang Fu, Li Erran Li, and Xiaolong Wang. 2026. [TIPS: Turn-level information-potential reward shaping for search-augmented LLMs](#). *arXiv preprint arXiv:2603.22293*.
- An Yang, Anfeng Li, Baosong Yang, and 1 others. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.
- Yelp Inc. 2026. [Yelp open dataset](#). Open dataset. Accessed 25 May 2026.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaocong Liu, Lingjun Liu, Xin Liu, and 1 others. 2025. [DAPO: An open-source LLM reinforcement learning system at scale](#). *arXiv preprint arXiv:2503.14476*.
- Dan Zhang, Sining Zhou, Min Cai, Fengzu Li, Lekang Yang, Wei Wang, Tianjiao Dong, Ziniu Hu, Jie Tang, and Yisong Yue. 2025a. [DataSciBench: An LLM agent benchmark for data science](#). *arXiv preprint arXiv:2502.13897*.
- Shaolei Zhang, Ju Fan, Meihao Fan, Guoliang Li, and Xiaoyong Du. 2025b. [DeepAnalyze: Agentic large language models for autonomous data science](#). *arXiv preprint arXiv:2510.16872*.
- Wenqi Zhang, Yongliang Shen, Weiming Lu, and Yuet-ing Zhuang. 2023. [Data-Copilot: Bridging billions of data and humans with autonomous workflow](#). *arXiv preprint arXiv:2306.07209*.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. [SGLang: Efficient execution of structured language model programs](#). In *Advances in Neural Information Processing Systems*.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [WebArena: A realistic web environment for building autonomous agents](#). In *International Conference on Learning Representations*.
- Yuhang Zhou, Lizhu Zhang, Yifan Wu, Jiayi Liu, Xiangjun Fan, Zhuokai Zhao, and Hong Yan. 2026. [Synthetic sandbox for training machine learning engineering agents](#). *arXiv preprint arXiv:2604.04872*.
- Yizhang Zhu, Shiyin Du, Boyan Li, Yuyu Luo, and Nan Tang. 2024. [Are large language models good statisticians?](#) In *Advances in Neural Information Processing Systems*.
- Zefang Zong, Dingwei Chen, Yang Li, Qi Yi, Bo Zhou, Chengming Li, Bo Qian, Peng Chen, and Jie Jiang. 2026. [AT²PO: Agentic turn-based policy optimization via tree search](#). *arXiv preprint arXiv:2601.04767*.

A Task Environment and Data Coverage

SciThèque is a collection of self-contained scientific discovery environments. Each environment gives the agent a natural-language hypothesis, task data, and an interactive analysis session. The context shown to the agent contains only the task description and data. The hidden materials used for verification are kept outside the policy context and are used only by the environment to judge progress and final correctness.

Environment snapshot. This environment snapshot contains 1,686 materialised environments across five modalities. Table 3 reports the modality distribution. The corpus is not limited to standard tabular prediction: it also includes time-series, spatial, sequence, and graph settings so that the trained agent must reuse the same scientific-interaction protocol across heterogeneous data forms.

Modality	Environments	Share
Tabular	1,024	60.7%
Time series	287	17.0%
Spatial	206	12.2%
Sequence	114	6.8%
Graph	55	3.3%
Total	1,686	100.0%

Table 3: Materialized Environment Distribution by Modality

Outcome and domain distribution. The corpus spans continuous, binary, count, multiclass, and other structured outcome types. Table 4 reports the outcome-kind distribution, with ordinal, modality-specific, and survival outcomes grouped as other structured outcomes. Table 5 reports the domain distribution, led by economics, medicine, biology, and earth science. The aggregate other-domain category contains additional scientific areas with distinct analysis conventions.

Outcome kind	Environments	Share
Continuous	716	42.5%
Binary	511	30.3%
Count	213	12.6%
Multiclass	209	12.4%
Other structured	37	2.2%
Total	1,686	100.0%

Table 4: Environment Distribution by Outcome Kind

Stress variants. The environment snapshot includes 504 stress variants derived from parent hypotheses. They perturb task settings while preserving the same interaction protocol and are counted separately from the aggregate corpus totals.

Domain	N	Domain	N
Economics	452	Medicine	334
Biology	325	Earth science	219
Engineering	127	Epidemiology	119
Other scientific domains	110		

Table 5: Environment Distribution by Scientific Domain

Data sources and licences. The environments are drawn from a mixture of general machine-learning repositories, open scientific time-series sources, biological databases, spatial and epidemiological data sources, economic indicators, and graph benchmarks (Kelly et al., 2023; Vanschoren et al., 2013; Federal Reserve Bank of St. Louis, 2026; The UniProt Consortium, 2025; Centers for Disease Control and Prevention, 2026a,b; U.S. Geological Survey, 2026; Yelp Inc., 2026; Godahewa et al., 2021; Szklarczyk et al., 2023; Hu et al., 2020). Table 6 reports the source distribution and the licence or access metadata recorded in the materialised catalogue. OpenML is listed with multiple licences because the source records licence metadata per dataset.

Intended use. The SciThèque environments, derived trajectories, and trained checkpoints are intended for research on training and evaluating scientific-discovery agents. They are not validated scientific findings, medical or public-health advice, economic recommendations, or operational decision-support systems. Derived artifacts retain the access conditions of their upstream sources: public-domain and permissively licensed sources support redistribution only within their stated terms, while sources with non-commercial or no-derivatives conditions are used as research inputs and are not relicensed for broader downstream deployment.

Privacy and content screening. The corpus is built from open scientific, public-service, and benchmark-style data sources rather than newly collected private communications. Source adapters materialise structured analysis variables and exclude direct identity fields when those fields are not needed for the scientific task. For example, the Yelp adapter keeps geographic coordinates, open-status, rating, review-count, and structured business attributes, but does not materialise business names, user identifiers, or review text. This screening is schema- and adapter-based, so it should not be read as a guarantee that every upstream source is free of sensitive or offensive content. The released task prompts and training traces expose only

Source	N	Recorded licence/access basis	Metadata scope
UCI	678	uci-redistributable	Source-level catalogue label
OpenML	475	public-domain, cc0-1.0, cc-by-4.0, cc-by-sa-4.0	Dataset-level OpenML metadata
FRED	151	public-domain	Source-level catalogue label
UniProt	114	cc-by-4.0	Source-level catalogue label
CDC FluView	65	public-domain	Source-level catalogue label
USGS	41	public-domain	Source-level catalogue label
CDC PLACES	36	public-domain	Source-level catalogue label
Yelp	40	cc-by-nc-nd-4.0	Source-level catalogue label
Monash	31	cc-by-4.0	Source-level catalogue label
STRING yeast	34	cc-by-4.0	Source-level catalogue label
OGB	21	mit	Source-level catalogue label

Table 6: Environment Counts and Metadata Scope by Data Source

task data and runtime observations; hidden verifier files, reference answers, and tolerance policies are kept outside the policy context and are separately checked by the leakage audit in Appendix B.

Data provenance and reproducibility. These counts are snapshot metadata for the environment corpus. Each task stores its source adapter, source dataset identifier, upstream provenance and licence metadata, modality, analysis template, prompt, hidden evidence graph, final checker, and data checksums.

Benchmark-overlap audit. The training corpus is audited against the evaluation snapshots used in Section 5.1. The audit compares SHA-256 hashes of materialised training data files with benchmark data files, then separately compares normalised source identifiers, task identifiers, benchmark directory names, and file stems. The audit covers 2,461 training data files with 991 unique file hashes and finds no direct overlap with the available benchmark snapshots: 0/165 DiscoveryBench files, 0/7 DABStep files, and 0/548 DataSciBench files. It also finds zero normalised source-identifier matches for all three benchmarks. The audit is scoped to exact file reuse and source-ID reuse, while broad topical similarity is expected across independently sourced scientific datasets.

Environment contract. Each environment separates three roles. The prompt specifies the scientific question and exposes the data. The hidden progress checker defines the ordered evidence needed before a final answer is admissible. The final-answer checker evaluates the submitted scientific claims against task-local reference values and tolerances. The checker is compiled from the same accepted reference analysis that materialises the task, while the tolerances and reference values remain outside the prompt and training transcript. This separation makes intermediate progress inspectable while preserving task-local final scoring.

Example task. A time-series environment can be

written as $u_j = (h_j, d_j, v_j)$. Here h_j asks whether an age-specific influenza-like-illness count helps predict total influenza-like-illness counts at a fixed weekly lag. The data component d_j is a weekly regional public-health series with a time index, a target count, age-specific count variables, patient totals, provider totals, and percentage-based illness measures. The verifier v_j checks that the trajectory inspects the series, assesses stationarity and sample size, fits an appropriate lagged time-series analysis, runs residual and stability diagnostics, performs a sensitivity check, and submits a conclusion whose significance bucket, effect-strength category, uncertainty verdict, and diagnostic outcomes match the task-local reference within the declared tolerances.

B Verifiability and Leakage Controls

The environment is designed so that intermediate reward comes from scientific state transitions rather than from matching hidden answer strings. A hidden evidence graph specifies which analysis steps are currently eligible, what observable evidence they must produce, and which downstream claims they support. The policy never sees this graph directly. Table 7 separates what is visible to the agent from what is used only by the verifier.

Evidence graph record. Each hidden evidence graph is stored as a task-local record with fields for task_id, node_id, process primitive, prerequisite node set, admissible operations, required evidence fields, supported derived claims, tolerance policy, hidden reference values, and visibility flag. Table 8 shows the hidden DAG for the same CDC FluView task whose prompt and accepted rollout appear in Tables 14 and 15. Node identifiers are shown here for auditability only; they are not included in the prompt or transcript.

Progress rule. A turn receives progress only when it produces new, inspectable evidence for exactly one eligible scientific step. Eligibility is defined by the current frontier of the hidden graph: a node

Surface	Visible to the agent	Used by the verifier
Task prompt	Hypothesis, available data, requested answer format and analysis constraints.	Prompt consistency checks and final-answer field expectations.
Data files	Input data available to the agent for analysis.	Runtime evidence produced by the agent from those data.
Intermediate observations	Execution output, errors, summaries, and feedback after each action.	Whether the action produced new evidence for exactly one currently eligible step.
Hidden evidence graph	Not visible.	Prerequisites, required evidence, supported claims, and progress state.
Reference answers	Not visible.	Final-answer scoring and task-local tolerance checks.

Table 7: Visibility split between agent context and verifier state

is available only after all of its prerequisites have been accepted. Repeating an already completed analysis, jumping to a future step before its prerequisites, bundling several steps into one action, or submitting the final answer prematurely leaves the hidden progress state unchanged. The terminal submission is credited only after the required intermediate evidence has been completed and the submitted claims pass the final checker.

Claim derivation. For claim-bearing steps, the verifier derives the relevant quantities from runtime evidence rather than trusting printed final literals. This prevents a trajectory from receiving progress simply by echoing a target value or by writing an answer-shaped string. A step must therefore both execute the appropriate analysis and expose enough structured evidence for the environment to validate the scientific quantity being claimed. Final checkers compare submitted answer fields against task-local references using hidden tolerance policies; the tolerance values are never serialized into prompts or exported SFT transcripts.

Expert review. The verifier mechanism was manually reviewed by 10 domain experts whose expertise covered the scientific domains represented in the environment corpus. The review focused on whether the evidence primitives, prerequisite ordering, diagnostic requirements, and acceptance criteria were appropriate for the corresponding domain and analysis template. Reviewers inspected the mechanism-level verifier specifications rather

than model outputs, so this audit checks the scientific plausibility of the progress rules rather than measuring rollout-level verifier precision.

Leakage controls. Prompts and training traces are screened for direct exposure of reference answers, hidden step identifiers, hidden checklist language, task-local tolerance values, or checker code. The final-answer checker and hidden progress specification are not copied into the training transcript. The materialisation audit scans prompts before manifest emission, and the transcript-conversion audit scans retained SFT traces before training-data export. A failed audit excludes the task or trace from the emitted corpus. These checks are especially important for supervised trajectories: a retained trace must demonstrate the analysis path, not merely reproduce values that were available to the trajectory generator.

Example verifier trace. Table 9 shows the verifier information associated with the accepted 10-step CDC FluView rollout in Table 15. The table reports the hidden verifier match and the visible evidence exposed to the policy after each action. During training, the policy observes the accepted flag and the observation text; the node labels are used only by the environment and are shown here for auditability.

Leakage exposure scan. The leakage scan checks the final RL prompt manifest and retained SFT traces for direct exposure of hidden verifier state outside the runtime evidence produced by executed analysis. The scan in Table 10 covers reference answer values in prompts or answer-format hints, task-local tolerance strings, hidden node identifiers, checker or gold-artifact markers, and generator-only markers.

C Trajectory Synthesis and Filtering

The cold-start corpus is built from successful environment interactions rather than from free-form teacher answers. Each retained trajectory must complete the hidden evidence graph, receive accepted progress at every required analysis step, and pass the final-answer checker. Table 11 summarizes the synthesis and filtering stages.

Retained corpus. The cold-start corpus contains 5,620 verified trajectories produced from accepted environment interactions.

Filtering boundary. Filtered traces are kept for diagnostics but excluded from training. The filter enforces whether a trajectory demonstrates the re-

Hidden node	Primitive	Prerequisite	Verifier evidence	Rollout turn
setup_time_series_data	Data setup	None	Loaded frame, role map, target, candidate variable, time index, and frequency.	1
prepare_series_frame	EDA summary	Setup	Sorted series frame with sample-size, shape, missingness, stationarity, and serial-dependence metrics.	2
build_lagged_design	Feature extraction	Series frame	Lag-1 design matrix with target, candidate, control, and lagged columns.	3
fit_temporal_model	Model fit	Lagged design	P-value and effect-size evidence for the selected temporal model.	4
check_stationarity	Diagnostic	Model fit	ADF stationarity evidence for the target series.	5
check_residual_whiteness	Diagnostic	Model fit	Ljung-Box and lag-1 autocorrelation evidence for residual dependence.	6
check_arch_effects	Diagnostic	Model fit	ARCH-effect diagnostic over fitted residuals.	7
estimate_temporal_ci	Uncertainty	Model fit	Confidence interval evidence for whether the effect overlaps the null.	8
check_convergence	Diagnostic	Model fit	Warning-aware convergence status for a fitted or alternative temporal model.	9
check_sensitivity	Robustness	Model fit	Main and alternative verdict comparison for method sensitivity.	10
submit	Terminal	Nodes 2–10	Submitted claims pass the final checker after required evidence is complete.	11

Table 8: Hidden evidence DAG for a CDC FluView time-series task

Turn	Verifier match	Decision	Visible evidence
1	setup_time_series_data	Accepted setup	Loaded 1,379 rows and 12 columns; printed the first rows.
2	prepare_series_frame	Accepted EDA summary	Reported rows, effective sample size, column count, and descriptive statistics.
3	build_lagged_design	Accepted feature extraction	Built a lag-1 design matrix with 1,378 rows and seven columns.
4	fit_temporal_model	Accepted model fit	Reported temporal model evidence with $p = 9.38 \times 10^{-26}$.
5	check_stationarity	Accepted diagnostic	Reported ADF stationarity evidence with $p = 1.41 \times 10^{-4}$.
6	check_residual_whiteness	Accepted diagnostic	Reported Ljung-Box residual evidence and lag-1 autocorrelation.
7	check_arch_effects	Accepted diagnostic	Reported ARCH-effect evidence with $p = 1.24 \times 10^{-266}$.
8	estimate_temporal_ci	Accepted uncertainty check	Reported the effect confidence interval, 5.25–5.41.
9	check_convergence	Accepted diagnostic	Reported warning-aware convergence status as yes.
10	check_sensitivity	Accepted robustness check	Reported that the main verdict was not method-sensitive across three alternatives.
11	submit	Accepted final submission	Final submission accepted after the required verifier state was complete.

Table 9: Verifier trace for the CDC FluView rollout

Surface	Audit target	Checked	Flagged
Prompt	Reference answer values	1,686	0
Prompt	Tolerance strings	1,686	0
Prompt	Hidden node identifiers	1,686	0
Prompt	Checker or gold markers	1,686	0
Prompt	Generator-only markers	1,686	0
SFT trace	Hidden node identifiers	5,620	0
SFT trace	Tolerance strings	5,620	0
SFT trace	Checker or gold markers	5,620	0
SFT trace	Generator-only markers	5,620	0

Table 10: Leakage exposure scan for prompts and retained SFT traces

quired scientific process rather than only whether it reaches the final answer.

D Training Details

Training has two stages. Cold-start supervised tuning teaches the solver the interaction format and rhythm of evidence-backed analysis. Agentic RL then optimizes the same protocol using environment turn-level progress signals. Table 12 reports the training and rollout hyperparameters.

Turn-level signal. For each prompt group, the environment records turns that complete new sci-

entific progress. A rollout receives positive signal on turns that advance the hidden evidence state; invalid, repeated, premature, or bundled turns receive no progress. The RL objective compares sibling rollouts at the same interaction position, emphasizing which action was better under the same prompt and comparable history length.

Filtering of uninformative groups. Prompt groups with no turn-level contrast are removed before optimization. This avoids updates where all siblings make identical progress, all fail, or all terminate without a distinguishable learning signal.

Stage	Purpose	Admission criterion
Evidence planning	Decompose the task into a prerequisite-ordered sequence of scientific analysis steps.	The hidden graph is well formed and all required claims are represented.
Trajectory scheduling	Create valid analysis orders or presentation variants for each environment.	The schedule respects prerequisites and ends with a final submission.
Execution	Run each scheduled analysis step in an isolated environment and collect observations.	The action executes within the turn budget and produces observable evidence.
Verification	Check each turn against the hidden progress rule and score the final answer.	Every required analysis step is accepted and the final answer passes.
Conversion	Keep only messages suitable for model training.	The transcript fits the token budget and passes leakage checks for hidden labels, hidden tolerances, reference values, and answer strings.

Table 11: Trajectory Synthesis and Filtering Pipeline for the Cold-Start Corpus

Hyperparameter	Value
<i>Cold-start supervised tuning</i>	
Backbone	Qwen3-14B
Corpus	SciDisco-produced
Epochs	1
Effective batch size	128
Learning rate	1×10^{-5}
Min learning rate	1×10^{-6}
Schedule	Cosine decay
Warm-up fraction	0.1
Weight decay	0.1
Context length	24,576 tokens
<i>Agentic RL</i>	
Initial checkpoint	SciDisco-14B-SFT
Learning rate	1×10^{-6}
Schedule	Constant
Rollout batch size	16
Global batch size	128
Rollout group size	8
Max turns	30
Per-turn generation cap	4,096 tokens
Prompt cap	8,192 tokens
Trajectory context cap	24,576 tokens
Sampling temperature	1.0
Top-p	0.95
Clip high	0.28
Clip low	0.2
Weight decay	0.1
Adam beta 1	0.9
Adam beta 2	0.98
Action timeout	60 s

Table 12: Training and Rollout Hyperparameters

The filter is therefore not a data-quality heuristic; it removes batches whose relative advantage is mathematically uninformative.

Execution budget. Each scientific analysis action runs under a fixed wall-clock timeout. The timeout bounds inefficient code and accidental infinite computations while allowing common statistical, graph, time-series, and spatial analyses to complete.

E DiscoPO Objective Details

The policy update assigns $A_{j,k,t}$ to the policy-generated tokens in turn t and excludes observation tokens from the loss. Let $\mathcal{I}_{j,k,t}$ denote the generated-token positions in that turn. For $i \in \mathcal{I}_{j,k,t}$, let y_i be the generated token and p_i its prefix. The likelihood ratio is:

$$\rho_i(\theta) = \frac{\pi_\theta(y_i | p_i)}{\pi_{\text{old}}(y_i | p_i)}. \quad (14)$$

The clipped ratio is:

$$\bar{\rho}_i(\theta) = \text{clip}(\rho_i(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}). \quad (15)$$

For each generated token, the clipped surrogate is:

$$\ell_i(\theta) = \min(\rho_i(\theta)A_{j,k,t}, \bar{\rho}_i(\theta)A_{j,k,t}). \quad (16)$$

DiscoPO optimizes the turn-averaged objective:

$$\begin{aligned} \bar{\ell}_{j,k,t}(\theta) &= \frac{1}{|\mathcal{I}_{j,k,t}|} \sum_{i \in \mathcal{I}_{j,k,t}} \ell_i(\theta), \\ \mathcal{L}_{\text{DiscoPO}}(\theta) &= -\mathbb{E}_{j,k,t} [\bar{\ell}_{j,k,t}(\theta)]. \end{aligned} \quad (17)$$

F Prompt and Rollout Trace

The interaction trace consists of a system prompt, a task prompt, alternating assistant actions and environment observations, and a terminal submission. Tables 13 and 14 give the system and task prompts used for the rollout in Table 15.

Rollout. Table 15 gives the same task’s rollout trajectory. The trace has 10 analysis actions and one final submission; each environment observation is accepted.

G Failure Cases

Rejected turns define the training-signal boundary. Table 16 summarizes the major failure modes guarded by the verifier during synthesis and RL interaction.

```

# Role
You are a data scientist investigating a statistical hypothesis on a scientific dataset. Each turn first writes one short
<thought>, then emits exactly one native action: <python>...</python> or <submit></submit>.

# Turn shape
Every assistant turn MUST follow this structure:
<thought>One or two sentences naming the next scientific analysis step.</thought>
<python>
# ordinary notebook-style scientific Python
</python>

When the analysis is complete, use this final action instead of Python:
<submit></submit>

# Rules
- Every assistant turn MUST open with a <thought>...</thought> block (1-2 sentences). The block exists for short orientation
  only.
- Use the thought to name the next analysis step in natural language, such as data setup, feature construction, model fit,
  diagnostic check, sensitivity analysis, or final submission.
- Put ordinary notebook-style scientific code inside <python>...</python>. Load the task data, inspect it, fit appropriate
  models or tests, and print concise intermediate results when useful.
- Kernel state persists across Python turns, so reuse variables from prior work instead of reloading everything each time.
- Use exactly one native action per turn. Do not combine Python and submit in the same turn.
- Use <submit></submit> only when the analysis is complete.

```

Table 13: System Prompt for Scientific Discovery Rollouts

```

# Hypothesis

Does `num_age_0` Granger-cause `num_ili` at lag 1?

# Data

Dataset: `cdc_flu_hhs10`

Files (load with pandas.read_parquet or pandas.read_csv):
- data/series.parquet

Time index column: `time_index`; frequency: `W-MON`

Value columns:
- `num_ili` (kind=continuous, role=target)
- `num_patients` (kind=continuous, role=candidate_iv)
- `num_providers` (kind=count, role=candidate_iv)
- `ili` (kind=continuous, role=candidate_iv)
- `wili` (kind=continuous, role=candidate_iv)
- `num_age_0` (kind=count, role=candidate_iv)
- `num_age_1` (kind=count, role=candidate_iv)
- `num_age_2` (kind=continuous, role=candidate_iv)
- `num_age_3` (kind=continuous, role=candidate_iv)
- `num_age_4` (kind=continuous, role=candidate_iv)
- `num_age_5` (kind=count, role=candidate_iv)

Dataset relative path: `data`

# Analysis expectations

Use Python to investigate the hypothesis step by step. Load the dataset files described above, inspect the relevant
variables, run appropriate statistical analyses and diagnostics, and finish when your scientific conclusion is ready.

```

Table 14: Task Prompt for the CDC FluView Rollout

Turn	Assistant action	Environment observation
1	<code><thought>I'll start by loading the dataset the task points to at `data`.</thought></code> <code><python> loaded the series frame and printed its first rows </python></code>	accepted: true Loaded 1379 rows × 12 columns printed the first five rows
2	<code><thought>I'll run code to summarize the dataset evidence needed for analysis.</thought></code> <code><python> sorted by time_index; computed rows, effective_n, and descriptive statistics </python></code>	accepted: true rows: 1379, effective_n: 1378, columns: 12 printed descriptive statistics
3	<code><thought>Before analysis continues, I need the design matrix.</thought></code> <code><python> built lagged_design with target, predictors, and lag-1 columns </python></code>	accepted: true lagged_design rows: 1378, cols: 7
4	<code><thought>I'll run the model fit and summarize the analysis evidence and inspect the output.</thought></code> <code><python> ran Granger and VAR fits; recorded p-value and effect-size evidence </python></code>	accepted: true n: 1379, lag: 1 fit_summary: p_value=9.38e-26
5	<code><thought>I'll run code to check the relevant diagnostic evidence.</thought></code> <code><python> ran augmented Dickey-Fuller stationarity check </python></code>	accepted: true adf_p: 0.000141
6	<code><thought>I'll check the relevant diagnostic evidence and inspect the output.</thought></code> <code><python> ran Ljung-Box residual-dependence check </python></code>	accepted: true ljung_p: 0, lag1_acf: 0.961
7	<code><thought>The data is ready, so I'll check the relevant diagnostic evidence.</thought></code> <code><python> ran ARCH-effect diagnostic </python></code>	accepted: true arch_p: 1.24e-266
8	<code><thought>I'll run the analysis code and inspect the resulting evidence and inspect the output.</thought></code> <code><python> computed confidence interval for the relevant effect estimate </python></code>	accepted: true ci: 5.25-5.41
9	<code><thought>This cell continues the notebook by running code to check the relevant diagnostic evidence.</thought></code> <code><python> fit ARIMA(1,0,0) and checked convergence warnings </python></code>	accepted: true converged: yes
10	<code><thought>The data is ready, so I'll run the analysis code and inspect the resulting evidence.</thought></code> <code><python> compared the main verdict with alternative method verdicts </python></code>	accepted: true sensitivity summary no, alternatives 3
11	<code><thought>The analysis is complete; time to submit.</thought></code> <code><submit></submit></code>	accepted: true message: final submission accepted

Table 15: Accepted Rollout Trace for a CDC FluView Time-Series Task

Failure mode	Typical cause	Training consequence
Premature final answer	The model submits before completing the required intermediate analyses.	No final-submission credit; the hidden progress state is unchanged.
Compound analysis	One turn bundles several scientific steps that should be separately verified.	The turn is rejected even if some computation is correct.
No new evidence	The action repeats an earlier result or prints unsupported claims without producing verifiable analysis output.	No progress credit is assigned.
Wrong prerequisite order	The model attempts a later analysis before the necessary setup or diagnostic evidence exists.	The action is treated as ineligible for progress.
Unverifiable claim	The output contains answer-like text, but the relevant value cannot be derived from runtime evidence.	The claim is ignored for progress and cannot support final credit.
Execution failure	The analysis errors, times out, or produces degenerate outputs.	The failed turn receives no progress, but later turns may still produce accepted evidence.
Final-answer mismatch	The intermediate process is complete, but the submitted values fail task-local tolerance checks.	The trajectory is not retained for supervised training and receives no successful terminal credit.

Table 16: Failure Modes and Training Consequences